

Activity No. 6	
SEARCHING TECHNIQUES	
Course Code: CPE010	Program: Computer Engineering
Course Title: Data Structures and Algorithms	Date Performed: 10/16/24
Section: CPE21S4	Date Submitted: 10/18/24
Name(s): Rio, Aries C.	Instructor: Ma'am Maria Rizette Sayo

6. Output

6-1

```

1  #include <iostream>
2  #include <cstdlib>
3  #include <ctime>
4  #include "nodes.h"
5  #include "searching.h"
6
7  const int max_size = 50;
8
9  int main() {
10     // Generate random values
11     int dataset[max_size];
12     srand(time(0));
13     for (int i = 0; i < max_size; i++) {
14         dataset[i] = rand() % 100;
15     }
16

```

Output

```

2 93 63 54 99 4 92 94 21 39 70 5 58 42 73 89 66 64 13 96 60 12 80 96 14 80 60 71 9
0 77 41 92 70 56 99 21 12 43 67 33 82 89 91 40 31 64 30 49 80 95

```

Explanation:

srand(time(0)): This line seeds the random number generator using the current time. This ensures that each run of the program will produce different random numbers.

rand(): This function generates a random integer.

dataset[i] = rand(): This line stores the generated random integer in the dataset array at index i. 6-2a

C searching.h

```
1  #ifndef SEARCHING_H
2  #define SEARCHING_H
3
4  #include <iostream>
5
6  template <typename T>
7  int linearSearch(T data[], int n, T item) {
8      for (int i = 0; i < n; i++) {
9          if (data[i] == item) {
10             std::cout << "Searching is successful\n";
11             return i; // Return the index
12         }
13     }
14     std::cout << "Searching is Unsuccessful\n";
15     return -1; // Item not found
16 }
17
18 template <typename T>
19 bool linearLS(Node<T>* head, T dataFind) {
20     Node<T>* current = head;
21     while (current != NULL) {
22         if (current->data == dataFind) {
23             std::cout << "Searching is successful\n";
24             return true;
25         }
26         current = current->next;
27     }
28     std::cout << "Searching is Unsuccessful\n";
29     return false;
30 }
```

Output

```
2 93 63 54 99 4 92 94 21 39 70 5 58 42 73 89 66 64 13 96 60 12 80 96 14 80 60 71 9
0 77 41 92 70 56 99 21 12 43 67 33 82 89 91 40 31 64 30 49 80 95
Searching for: 80
Searching is successful
Searching is successful
```

Explanation:

Successful Search: If the randomly generated dataset contains the item being searched for, the output will indicate "Searching is successful" along with the index of the item.

Unsuccessful Search: If the item is not found in the dataset, the output will show "Searching is Unsuccessful". Performance: The linear search in an array runs in $O(n)$ time complexity, making it less efficient for large datasets.

```
// Create linked list for linear search
```

```
Node<char>* name1 = new_node( 'R' );
```

```
Node<char>* name2 = new_node( 'o' );
```

```
Node<char>* name3 = new_node( 'm' );
```

```
Node<char>* name4 = new_node( 'a' );
```

```
Node<char>* name5 = new_node( 'n' );
```

```
// Link nodes
```

```
name1->next = name2;
```

```
name2->next = name3;
```

```
name3->next = name4;
```

```
name4->next = name5;
```

```
Linked list contents: R o m a n
```

Output Explanation:

Character Search: Searching for a character (e.g., 'n' in the name "Roman") will yield "Searching is successful" if found, or "Searching is Unsuccessful" if not.

Traversal: The function traverses each node in the linked list until it finds the desired character or reaches the end of the list.

Time Complexity: Similar to the array, the linear search on a linked list also runs in $O(n)$ time complexity. 6-3a

```

8  template <typename T>
9  Node<T>* binarySearch(Node<T>* head, T item) {
10     Node<T>* start = head;
11     Node<T>* end = NULL;
12
13     do {
14         Node<T>* mid = getMiddle(start, end);
15
16         if (mid == NULL) return NULL;
17
18         if (mid->data == item) {
19             std::cout << "Search element is found!\n";
20             return mid;
21         } else if (mid->data < item) {
22             start = mid->next;
23         } else {
24             end = mid;
25         }
26     } while (end == NULL || end != start);
27
28     std::cout << "Search element is not found\n";
29     return NULL;
30 }

```

Output

```

92 60 44 71 69 60 55 17 96 7 43 56 86 94 28 0 71 24 95 68 42 67 34 97 31 30 1 25 7
5 3 42 20 63 38 91 85 99 46 54 95 53 49 4 91 43 84 43 14 60 38
Searching for: 85
Searching is successful
Linked list contents: R o m a n
Searching is successful
Binary search linked list: 2 6 69 52 37
Binary searching for: 50
Search element is not found

```

Explanation:

Successful Search: If the item is found, the output will show "Search element is found!" indicating that the search was successful. The linked list traversal to find the middle element uses a two-pointer technique, ensuring efficiency.

Unsuccessful Search: If the item is not in the list, the output will indicate "Search element is not found". The algorithm effectively reduces the search space by half with each iteration, providing $O(\log n)$ time complexity in a well-structured linked list.


```

// Display linked list for binary search
temp = head;
cout << "Binary search linked list: ";
while (temp != nullptr) {
    cout << temp->data << " ";
    temp = temp->next;
}
cout << endl;

// Perform binary search
int binarySearchValue = 50; // Example value
cout << "Binary searching for: " << binarySearchValue << endl;
binarySearch(head, binarySearchValue);

return 0;
}

```

```

48 template <typename T>
49 Node<T>* binarySearch(Node<T>* head, T item) {
50     Node<T>* start = head;
51     Node<T>* end = NULL;
52
53     do {
54         Node<T>* mid = getMiddle(start, end);
55
56         if (mid == NULL) return NULL;
57
58         if (mid->data == item) {
59             std::cout << "Search element is found!\n";
60             return mid;
61         } else if (mid->data < item) {
62             start = mid->next;
63         } else {
64             end = mid;
65         }
66     } while (end == NULL || end != start);
67
68     std::cout << "Search element is not found\n";
69     return NULL;
70 }
71
72 #endif // SEARCHING_H
73

```

Output

```
Binary search linked list: 2 6 69 52 37
Binary searching for: 50
Search element is not found
```

Explanation:

Node and List Creation: Creates a sorted linked list with values 0, 10, 20, ..., 90.

Binary Search Implementation: `getMiddle` finds the middle node using the two-pointer approach. `binarySearch` uses the middle node to check if the key is found or to adjust the search range.

Test Cases: Searches for three values: 50. Outputs whether each value was found in the linked list.

7. Supplementary Activity

Problem 1

```
1  #include <iostream>
2
3  using namespace std;
4
5  // Linear Search in Array
6  int linearSearchArray(int arr[], int size, int key) {
7      int comparisons = 0;
8      for (int i = 0; i < size; i++) {
9          comparisons++;
10         if (arr[i] == key) {
11             return comparisons; // Return number of comparisons
12         }
13     }
14     return comparisons; // Return comparisons even if not found
15 }
16
17 // Node structure for Linked List
18 struct Node {
19     int data;
20     Node* next;
21 };
22
23 // Function to create a new node
24 Node* new_node(int data) {
25     Node* node = new Node();
26     node->data = data;
27     node->next = nullptr;
28     return node;
29 }
30
31 // Linear Search in Linked List
32 int linearSearchLinkedList(Node* head, int key) {
33     int comparisons = 0;
34     Node* current = head;
35     while (current != nullptr) {
36         comparisons++;
37         if (current->data == key) {
38             return comparisons; // Return number of comparisons
39         }
40         current = current->next;
41     }
42     return comparisons;
43 }
```

```

44
45 v int main() {
46     // Given list
47     int arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
48     int size = sizeof(arr) / sizeof(arr[0]);
49     int key = 18;
50
51     // Sequential search in array
52     int comparisonsArray = linearSearchArray(arr, size, key);
53     cout << "Array comparisons for key " << key << ": " << comparisonsArray << endl;
54
55     // Create linked list
56     Node* head = new_node(15);
57     head->next = new_node(18);
58     head->next->next = new_node(2);
59     head->next->next->next = new_node(19);
60     head->next->next->next->next = new_node(18);
61     head->next->next->next->next->next = new_node(0);
62     head->next->next->next->next->next->next = new_node(8);
63     head->next->next->next->next->next->next->next = new_node(14);
64     head->next->next->next->next->next->next->next->next = new_node(19);
65     head->next->next->next->next->next->next->next->next->next = new_node(14);
66
67     // Sequential search in linked list
68     int comparisonsLinkedList = linearSearchLinkedList(head, key);
69     cout << "Linked list comparisons for key " << key << ": " << comparisonsLinkedList <<
endl;
70
71     return 0;
72 }
73

```

Output

```

Array comparisons for key 18: 2
Linked list comparisons for key 18: 2

```

Answer

Array: The search found '18' in 2 comparisons.

Linked List: The search also found '18' in 2 comparisons.

Problem 2

```
#include <iostream>

using namespace std;

// Count occurrences in Array
int countOccurrencesArray(int arr[], int size, int key) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            count++;
        }
    }
    return count; // Return the count of occurrences
}

// Count occurrences in Linked List
int countOccurrencesLinkedList(Node* head, int key) {
    int count = 0;
    Node* current = head;
    while (current != nullptr) {
        if (current->data == key) {
            count++;
        }
        current = current->next;
    }
    return count; // Return the count of occurrences
}

int main() {
```



```

// Given list
int arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
int size = sizeof(arr) / sizeof(arr[0]);
int key = 18;

// Count occurrences in array
int countArray = countOccurrencesArray(arr, size, key);
cout << "Occurrences of " << key << " in array: " << countArray << endl;

// Create linked list (same as before)
Node* head = new_node(15);
head->next = new_node(18);
head->next->next = new_node(2);
head->next->next->next = new_node(19);
head->next->next->next->next = new_node(18);
head->next->next->next->next->next = new_node(0);
head->next->next->next->next->next->next = new_node(8);
head->next->next->next->next->next->next->next = new_node(14);
head->next->next->next->next->next->next->next->next = new_node(19);
head->next->next->next->next->next->next->next->next->next = new_node(14);

// Count occurrences in linked list
int countLinkedList = countOccurrencesLinkedList(head, key);
cout << "Occurrences of " << key << " in linked list: " << countLinkedList << endl;

return 0;
}

```

Output

```

Occurrences of 18 in array: 2
Occurrences of 18 in linked list: 2

```

Answer

Both methods found 2 occurrences of the key '18'.

Problem 3

main.cpp > ...

```
1  #include <iostream>
2
3  using namespace std;
4
5  // Binary Search Function
6  int binarySearch(int arr[], int size, int key) {
7      int low = 0, high = size - 1;
8
9      while (low <= high) {
10         int mid = low + (high - low) / 2;
11         if (arr[mid] == key) {
12             return mid; // Key found
13         } else if (arr[mid] < key) {
14             low = mid + 1; // Search right half
15         } else {
16             high = mid - 1; // Search left half
17         }
18     }
19     return -1; // Key not found
20 }
21
22 int main() {
23     int arr[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
24     int size = sizeof(arr) / sizeof(arr[0]);
25     int key = 8;
26
27     int result = binarySearch(arr, size, key);
28     if (result != -1) {
29         cout << "Element " << key << " found at index: " << result << endl;
30     } else {
31         cout << "Element " << key << " not found." << endl;
32     }
33
34     return 0;
35 }
```

Output

```
Element 8 found at index: 3
```

Answer

First Iteration: Low = 0, High = 9 (array indices) Mid = 4 (value 11) Key < Mid value → Search right half. Second Iteration: Low = 0, High = 3 Mid = 1 (value 5) Key > Mid value → Search right half. Third Iteration: Low = 2, High = 3 Mid = 2 (value 6) Key > Mid value → Search right half. Fourth Iteration: Low = 3, High = 3 Mid = 3 (value 8) Key = Mid value → Found!

Problem 4

```

1  #include <iostream>
2
3  using namespace std;
4
5  // Recursive Binary Search Function
6  int recursiveBinarySearch(int arr[], int low, int high, int key) {
7      if (low > high) {
8          return -1; // Key not found
9      }
10
11     int mid = low + (high - low) / 2;
12     if (arr[mid] == key) {
13         return mid; // Key found
14     } else if (arr[mid] < key) {
15         return recursiveBinarySearch(arr, mid + 1, high, key); // Search right half
16     } else {
17         return recursiveBinarySearch(arr, low, mid - 1, key); // Search left half
18     }
19 }
20
21 int main() {
22     int arr[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
23     int size = sizeof(arr) / sizeof(arr[0]);
24     int key = 8;
25
26     int result = recursiveBinarySearch(arr, 0, size - 1, key);
27     if (result != -1) {
28         cout << "Element " << key << " found at index: " << result << endl;
29     } else {
30         cout << "Element " << key << " not found." << endl;
31     }
32
33     return 0;
34 }

```

Output

```
Element 8 found at index: 3
```

Answer

The recursive binary search also successfully found the key '8' at index 3, similar to the iterative method.

8. Conclusion

In this activity, I learned how to implement and analyze linear and binary search algorithms in C++. I gained a better understanding of their efficiency and how to apply them to linked lists. While I successfully implemented these searches, I faced challenges with binary search due to the need for ordered values in linked lists. I also looked at how many comparisons were needed for sequential search and used diagrams to show how binary search works. Overall, I completed the tasks well but need to improve on handling edge cases and practicing more with recursive algorithms.

9. Assessment Rubric